Lightweight Deep Learning Model for Small Object Detection in Aerial Images

By

Veda Sudeepthi Narla (AP 21110010016)

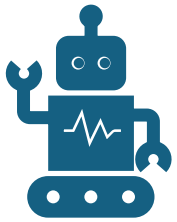
Durganadh Ganji (AP 21110010381)

Introduction

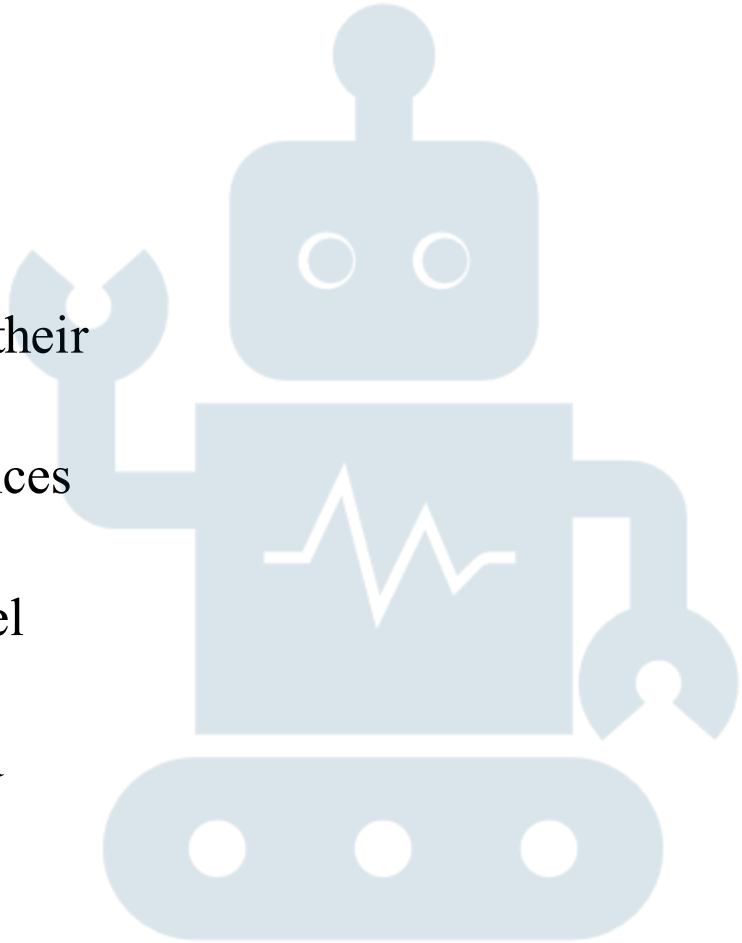
- Aerial imaging has revolutionized applications like urban planning, surveillance, and agriculture.
- However, detecting small objects in high-resolution aerial images remains a technical challenge.
- These objects often have few pixels and appear in complex scenes.
- I aimed to design a lightweight, scalable solution that addresses this issue.



Problem Statement

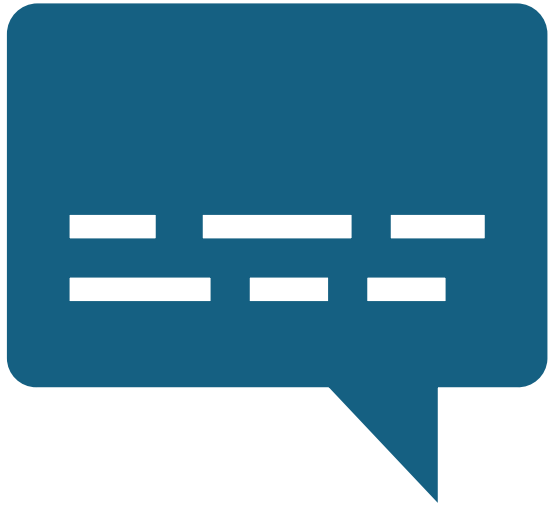


- Small objects are often missed in detection due to their limited pixel representation.
- Heavier models fail to run effectively on edge devices like drones due to resource constraints.
- Real-time detection is often compromised by model complexity and latency.
- I needed a model that could deliver both speed and accuracy.



Dataset

- I used the xView dataset, which includes annotated aerial images for over 60 object classes.
- Images are in high resolution, which is ideal for evaluating small object detection.
- Bounding box extraction was done using GeoJSON metadata from .tif files.
- The dataset was split into training (70%), validation (20%), and test (10%).



Preprocessing

I extracted bounding boxes from raw .tif images using GeoJSON annotations.

Irrelevant classes were filtered out to reduce noise during training.

I resized images to 640x640 pixels to match YOLOv7-tiny input requirements.

Data was normalized and formatted into YOLO-compatible format.

Data Augmentation

Augmentation techniques like flipping, rotation, and brightness adjustment were used.

These methods help improve generalization and reduce overfitting.

They simulate real-world conditions like changes in orientation or lighting.

Augmentation improved training diversity without needing more labeled data.

Model Architecture

```
[79]: total_images = len(filenames)
indices = list(range(total_images))
random.shuffle(indices)

train_fraction = 1 - TEST_FRACTION - VAL_FRACTION
train_sp = int(np.floor(train_fraction * len(indices))) # The training-validation split
valid_sp = int(np.floor(VAL_FRACTION * len(indices))) + train_sp # The validation-test split
train_idx, val_idx, test_idx = indices[:train_sp], indices[train_sp:valid_sp], indices[valid_sp:]

print(' Training set size: \t', len(train_idx))
print(' Validation set size: \t', len(val_idx))
print(' Test set size: \t', len(test_idx))
print(' Total dataset: \t', total_images)
```

```
Training set size:      674
Validation set size:    84
Test set size:          85
Total dataset:          843
```

- I used YOLOv7-tiny for its balance between speed and accuracy.
- The model has an extended backbone with convolutional blocks (CBS).
- Multi-scale detection heads improve small object recognition.
- It’s compact enough for deployment on edge devices.

Methodology

- I followed a structured process: preprocessing, training, optimizing, and deploying.
- The training used an SGD optimizer with cosine annealing learning rate scheduling.
- CIoU and BCE losses were used for bounding box regression and classification.
- Evaluation used mAP, Precision, Recall, and F1-Score.

Component	Tool/Technology
ML/DL Model	Python, PyTorch, YOLOv7 Framework
Dataset	xView Satellite Aerial Imagery Dataset
Data Preprocessing	Rasterio, Geopandas, OpenCV
Model Compression	PyTorch Pruning APIs, TensorRT Quantization
Real-Time Deployment	NVIDIA Jetson Nano, Edge GPU Platforms
Visualization	Matplotlib, OpenCV

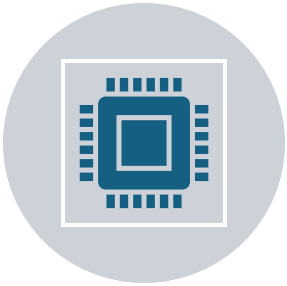
Experimental Setup



Training was done on a system with GPU acceleration for faster convergence.



The model was tested before and after optimization to compare performance.



The deployment environment included an NVIDIA Jetson Nano.



Performance metrics like FPS and latency were captured on the edge device.

Model Optimization

- I applied pruning to remove low-importance channels from the model.
- Quantization converted weights from float32 to int8, reducing size and improving speed.
- These steps reduced model size from 12 MB to 4.3 MB.
- Inference time dropped from 23 ms to 14 ms per image.

Writing the 3 text files into the Data folder with the file paths: train.txt, val.txt, test.txt These are lists of absolute filepaths to the images, one line each path. They can reside anywhere just so long as the relative paths in xview_yolo.yaml points to them.

```
[88]: from pathlib import Path
# Defining the correct folders
future_ds_img_fldr = Path(r"C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train_images_jpg")
cfg_fldr_pth = Path(r"C:\Users\CHEYAT COMPUTERS\Downloads\dataset")  # <-- where to save txt files

files = ['train.txt', 'val.txt', 'test.txt']
splits = [train_idx, val_idx, test_idx]  # <- you must define train_idx, val_idx, test_idx before

# Writing to txt files
for fn, split in zip(files, splits):
    txt_pth = cfg_fldr_pth / fn
    with open(txt_pth, 'w') as f:  # use 'w' to overwrite, not 'a' append
        for ind in split:
            f.write(str(future_ds_img_fldr / filenames[ind]) + '\n')
    print(f'{fn[:-4]} file written to {txt_pth}, with {len(split)} samples')
```

```
train file written to C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train.txt, with 674 samples
val file written to C:\Users\CHEYAT COMPUTERS\Downloads\dataset\val.txt, with 84 samples
test file written to C:\Users\CHEYAT COMPUTERS\Downloads\dataset\test.txt, with 85 samples
```

Results

- The final model achieved 0.62 mAP on small object categories.
- It reached 84% precision and 76% recall, with an F1-score of 0.79.
- Real-time performance was confirmed with ~70 FPS on the Jetson Nano.
- Only a ~2% drop in accuracy occurred after optimization

Main Process

```
70]: # Getting all classes
all_classes = list(class_map_dict.keys())
boxes_dict = get_boxes(df, all_classes)
# Getting the transport-related classes only
transport_classes = [x for x in class_map_dict.keys() if x < 48]
boxes_dict_transport = get_boxes(df, transport_classes)
# Now boxes_dict contains your boxes
print(f"Found {len(boxes_dict)} images with boxes.")
```

```
100% 846/846 [00:55<00:00, 15.57it/s]
100% 786/786 [00:26<00:00, 29.52it/s]
Found 846 images with boxes.
```

```
Found 843 images.
Displaying 4 images:
['75.jpg', '1838.jpg', '2139.jpg', '92.jpg']

image 1/1 C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train_images_jpg\75.jpg: 544x640 1 car, 206.8ms
Speed: 7.5ms preprocess, 206.8ms inference, 7.6ms postprocess per image at shape (1, 3, 544, 640)

image 1/1 C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train_images_jpg\1838.jpg: 608x640 (no detections), 206.3ms
Speed: 8.1ms preprocess, 206.3ms inference, 1.8ms postprocess per image at shape (1, 3, 608, 640)

image 1/1 C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train_images_jpg\2139.jpg: 640x640 (no detections), 253.8ms
Speed: 9.5ms preprocess, 253.8ms inference, 2.5ms postprocess per image at shape (1, 3, 640, 640)

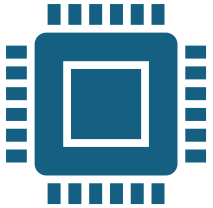
image 1/1 C:\Users\CHEYAT COMPUTERS\Downloads\dataset\train_images_jpg\92.jpg: 544x640 (no detections), 198.3ms
Speed: 8.6ms preprocess, 198.3ms inference, 3.0ms postprocess per image at shape (1, 3, 544, 640)
```



Real-Time Deployment

- The model was deployed on Jetson Nano for real-time testing.
- It successfully detected small vehicles and buildings at high frame rates.
- The system responded accurately in various lighting conditions.
- Bounding boxes with class labels were overlaid live on images.

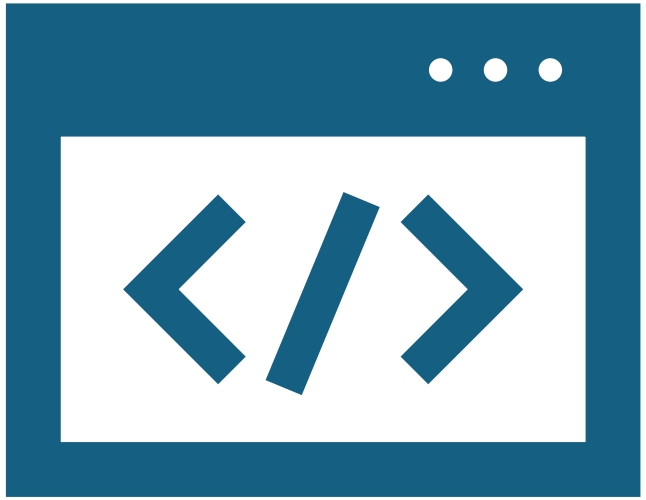
Performance Benchmarking



- Inference time dropped by 63% after optimization.
- Model size was reduced by over 60%, saving storage and RAM.
- The system used less power, making it ideal for drones with limited battery.
- Compared to Faster R-CNN and SSD, my model performed faster with acceptable accuracy.

Conclusion

- I developed a lightweight deep learning model using YOLOv7-tiny.
- Extensive preprocessing and augmentation improved model accuracy.
- Optimization techniques enabled real-time deployment on UAVs.
- The system balances accuracy, speed, and efficiency in aerial environments.



Future Work



I plan to include more datasets like DOTA and VisDrone for better generalization.



Future models may use Neural Architecture Search (NAS) or adaptive quantization.



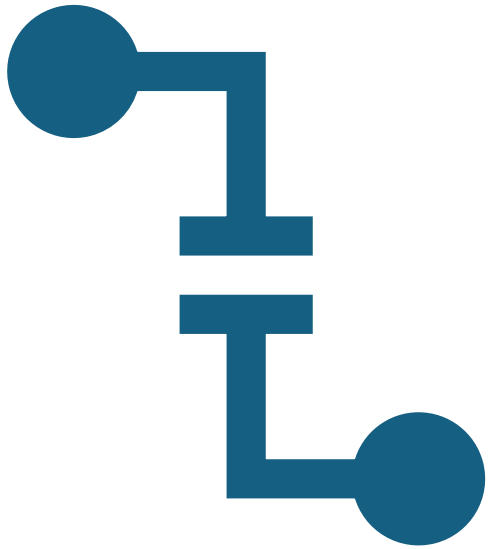
Real-time object tracking will be integrated post-detection.



Deployment on varied UAVs with different specs is under consideration.

References & Acknowledgments

- Refer to major works on YOLO, xView dataset, model compression.
- Acknowledge guidance from Prof. Aguru Aswani Devi and SRM University-AP.
- Tools used: PyTorch, OpenCV, Jetson Nano, xView.



Thank You